

AULA 1

JavaScript na prática

Sintaxe, exibição, coleta de dados e operações. Direto no terminal.

```
> console.log('Olá, turma!')  
Olá, turma!
```

VISÃO GERAL

O plano de hoje

- 01 Abertura e ambiente
- 02 Regras da linguagem e sintaxe
Pausa
- 03 Exibição de informações
- 04 Coleta de informações: bibliotecas de prompt
Pausa
- 05 Operações com as informações
- 06 Mini-projeto: recibo de compra
- 07 Fechamento

OBJETIVOS

Ao final da aula você vai conseguir

Escrever

código com a sintaxe
e as regras da
linguagem.

Exibir

informações
formatadas na tela.

Coletar

dados do teclado,
com o tipo certo.

Operar

com números, textos
e valores lógicos.

E juntar tudo em um programa: um recibo de compra no terminal.

NIVELAMENTO

Quem já escreveu JavaScript?

E quem já programou em outra linguagem?

ABERTURA

O que é JavaScript

1995

Criada por Brendan Eich na Netscape, em cerca de 10 dias.

ECMAScript

O padrão oficial. Uma versão nova por ano desde 2015.

2009

O Node.js leva o JavaScript para fora do navegador.

Linguagem dinâmica, de tipagem fraca e multiparadigma. E sem parentesco com Java.

ABERTURA

Onde o JavaScript roda

Navegador

Chrome, Firefox e Safari.
Manipula páginas (DOM).

Terminal e servidor

Node.js, Deno e Bun. Arquivos,
rede, scripts.

Outros lugares

Apps desktop e mobile, nuvem,
dispositivos.

Hoje: só o terminal. **A linguagem é a mesma;** muda o ambiente ao redor.

Motores: V8 (Chrome, Node.js) · SpiderMonkey (Firefox) · JavaScriptCore (Safari, Bun)

AMBIENTE

Rodando JavaScript sem página web

```
$ node --version  
v22.x.x  
$ node  
> 2 + 3  
5  
> .exit  
$ node arquivo.js  
$ node --watch arquivo.js
```

REPL: testa ideias de uma linha.

Arquivo: o modo do dia a dia.

--watch: reexecuta ao salvar.

AMBIENTE

Checklist de ambiente

1. Instale o **Node.js LTS**
2. Abra a pasta **aula1-sintaxe** no VS Code
3. Abra o terminal: Terminal > Novo Terminal
4. Confira com **node --version**

Sem permissão para instalar? Use StackBlitz ou Replit.

```
$ winget install OpenJS.NodeJS.LTS
$ node --version
$ cd aula1-sintaxe
$ node exemplos/01-ola.js
Olá, mundo!
Somando: 5
```

AMBIENTE

Primeiro programa

```
console.log('Olá, mundo!');  
console.log('Somando:', 2 + 3);
```

console.log exibe na tela.

Vários argumentos, separados por vírgula.

A conta **2 + 3** é feita antes de exibir.

Anatomia de um programa

```
// Isto é um comentário: o Node ignora
const nome = 'Ana';           // toda instrução termina com ;
let total = 10 + 5;

/* Comentário
   de várias linhas */
console.log(nome, total);      // executa de cima para baixo
```

Instrução termina com ;

Comentários: // e /* */

Roda de **cima para baixo**.

Regras básicas da linguagem

Maiúsculas

nome e **Nome** são nomes diferentes.

Ponto e vírgula

Cada instrução termina com **;**.

Blocos

Chaves **{ }** agrupam instruções.

Comentários

// uma linha, **/* */** várias.

Texto

Aspas simples, duplas ou crases.

Ordem

O código roda de cima para baixo.

REGRAS E SINTAXE

Nomes válidos (identificadores)

Nome	Válido?	Motivo
idade	sim	só letras
_total	sim	pode começar com _
\$preco	sim	pode começar com \$
nomeCompleto	sim	camelCase: a convenção
1nota	não	não começa com dígito
meu nome	não	sem espaços
nome-completo	não	o hífen é o operador -
class	não	palavra reservada

REGRAS E SINTAXE

Palavras reservadas

const

let

var

function

return

if

else

for

while

do

switch

case

break

continue

class

new

this

typeof

import

export

await

async

try

catch

throw

true

false

null

undefined

Fazem parte da sintaxe e **não podem ser usadas como nomes** de variáveis.

ATIVIDADE 1 · SÓ INDIQUE A RESPOSTA

Nomes válidos ou inválidos?

total2

2total

valor-final

valorFinal

_temp

\$valor

let

nome completo

PI

preço

1. Diga quais nomes são **válidos** e quais são **inválidos**
2. Explique o motivo de cada nome inválido
3. Em caso de dúvida, teste no REPL (**node**)

Desafio: corrija os nomes inválidos.

REGRAS E SINTAXE

Variáveis: const, let e var

```
const linguagem = 'JavaScript';    // não muda
let alunos = 20;                    // pode mudar
alunos = alunos + 1;

linguagem = 'TypeScript';           // TypeError!
var antigo = 'evite';               // escopo de função
```

const por padrão.

let só se for reatribuir.

var: evite (escopo de função).

REGRAS E SINTAXE

Declarar, atribuir e reatribuir

```
let idade;           // declarada: vale undefined
idade = 20;          // atribuição
let ano = 2026;       // declaração + valor inicial
ano = ano + 1;        // reatribuição
const PI = 3.14159;   // const exige valor inicial
```

Declarar: criar a variável.

Atribuir: dar um valor.

const exige valor na declaração.

REGRAS E SINTAXE

Tipos primitivos

string

'texto'

number

42 3.14

boolean

true false

null

vazio

undefined

sem valor

bigint

123n

symbol

Symbol()

object

{ } [] função

```
typeof 'oi'      // "string"
typeof 42        // "number"
typeof null      // "object" (bug histórico)
```

Tipagem dinâmica

```
let valor = 42;  
typeof valor          // "number"  
  
valor = 'quarenta e dois';  
typeof valor          // "string"  
  
valor = true;  
typeof valor          // "boolean"
```

O tipo pertence ao **valor**.

typeof mostra o tipo atual.

A variável pode mudar de tipo sem aviso.

Variáveis e tipos

1. Crie um arquivo novo (por exemplo, **atividade2.js**)
2. Duas **constantes**: **nomeCurso** (texto) e **cargaHoraria** (número)
3. Duas **variáveis**: **vagasRestantes** (número; depois reatribua um valor menor) e **turmaAberta** (boolean)
4. Mostre cada valor junto com o seu **typeof**

Desafio: reatribua **nomeCurso** e leia a mensagem de erro.

Lendo mensagens de erro

ReferenceError

Usou algo que não existe, ou com maiúscula errada.

TypeError

Ex.: Assignment to constant variable. Reatribuiu uma const.

SyntaxError

Escreveu algo fora das regras: nome inválido, faltou).

TypeError

Ex.: Cannot read properties of undefined. Usou . em algo vazio.

Leia de cima para baixo: o primeiro trecho aponta o arquivo e a linha.

EXERCÍCIO 1 · REPRODUZA NO COMPUTADOR E TESTE

Corrija os erros

```
const nomeAluno = 'Ana';  
let 1nota = 8;  
let nota2 = 9,5;  
const escola = 'IFSP';  
escola = 'USP';  
console.log(NomeAluno, nota2);
```

1. Reproduza o código acima em um arquivo novo
2. Rode com **node**, leia o erro e corrija a linha indicada
3. Repita até imprimir o nome e a nota sem erros

Desafio: depois de corrigir, invente um erro novo e leia a mensagem.

INTERVALO

Pausa

Na volta: exibição de informações

EXIBIÇÃO

console.log

```
console.log('Texto', 42, true);    // Texto 42 true
console.log(null, undefined);      // null undefined
console.log(3 + 4);                 // 7
console.log('3 + 4');               // 3 + 4
console.log();                      // (linha em branco)
```

Vários argumentos, separados por espaço.

Aspas: é **texto**; sem aspas: é **conta**.

Sem argumentos: linha em branco.

EXIBIÇÃO

Outros métodos do console

```
console.log('normal');  
console.info('informação');  
console.warn('aviso');      // vai para o stderr  
console.error('erro');      // vai para o stderr  
console.table([{ nome: 'Ana', idade: 20 }, { nome: 'Bia', idade: 22 }]);
```

info/warn/error também exibem.

warn e **error** usam o stderr.

table mostra dados em tabela.

EXIBIÇÃO

Template literals

```
const nome = 'Ana';  
const idade = 20;  
  
console.log(`Olá, ${nome}!`);           // Olá, Ana!  
console.log(`Em 5 anos: ${idade + 5}`); // Em 5 anos: 25  
console.log(`${nome.toUpperCase()} - ${idade}`); // ANA - 20  
console.log(`Olá, ' + nome + '!`);      // concatenação: igual
```

Crases + **`${expressão}`**.

Aceita contas e métodos dentro.

Aspas simples **não** interpolam.

ATIVIDADE 3 · REPRODUZA NO COMPUTADOR E TESTE

Apresentação

```
const nome = 'Ana';  
const idade = 20;  
const cidade = 'São Paulo';
```

1. Reproduza os dados acima em um arquivo novo
2. Mostre: Olá, meu nome é Ana, tenho 20 anos e moro em São Paulo.
3. Faça de 3 jeitos: template literal, concatenação com **+** e vários argumentos
4. Depois mostre a frase em duas linhas, usando **\n**

Desafio: mostre os três dados com **console.table**.

EXIBIÇÃO

Caracteres especiais (escapes)

```
console.log('Linha 1\nLinha 2');           // duas linhas
console.log('Col A\tCol B');               // tabulação
console.log('Caminho: C:\\aula\\js');      // Caminho: C:\aula\js
console.log('It\'s aqui');                 // It's aqui
console.log("Ele disse: \"oi\"");          // Ele disse: "oi"
console.log("It's aqui");                  // aspas duplas: sem escape
```

| **\n** nova linha, **\t** tabulação.

| **** imprime uma barra.

| Ou troque o tipo de aspas.

EXIBIÇÃO

Especificadores de formato

```
console.log('%s tem %d anos', 'Ana', 20); // Ana tem 20 anos
console.log('Pi = %f', 3.14159);           // Pi = 3.14159
console.log('Inteiro: %i', 3.9);           // Inteiro: 3
console.log('Dados: %j', { a: 1 });        // Dados: {"a":1}
console.log('%d%% de desconto', 15);      // 15% de desconto
```

%s texto, **%d** número, **%i** inteiro.

%f decimal, **%j** JSON.

%% imprime o símbolo %.

EXIBIÇÃO

Formatando números

```
const preco = 1234.5;
const brl = { style: 'currency', currency: 'BRL' };

preco.toFixed(2) // "1234.50" (texto!)
preco.toLocaleString('pt-BR') // "1.234,5"
preco.toLocaleString('pt-BR', brl) // "R$ 1.234,50"
(0.256).toLocaleString('pt-BR', { style: 'percent' }) // "26%"
```

toFixed devolve texto.

toLocaleString usa o padrão brasileiro.

Formate só para **exibir**.

Quatro formas de mostrar um número

```
const preco = 1499.9;  
const desconto = 0.15;
```

1. Reproduza os dados acima em um arquivo novo
2. Mostre o preço com **toFixed(2)** e confira o **typeof** do resultado
3. Mostre o preço no padrão **pt-BR** e depois em reais (**currency, BRL**)
4. Mostre o desconto em percentual (**style: 'percent'**)

Desafio: mostre o preço com 3 casas decimais no padrão brasileiro.

EXIBIÇÃO

Alinhando colunas

```
console.log('Item'.padEnd(12) + 'Preço'.padStart(10));  
console.log('-'.repeat(22));  
console.log('Caderno'.padEnd(12) + '18,90'.padStart(10));  
console.log('Caneta'.padEnd(12) + '3,50'.padStart(10));
```

Item	Preço

Caderno	18,90
Caneta	3,50

EXERCÍCIO 2 · REPRODUZA NO COMPUTADOR E TESTE

Ficha do produto

```
const produto = 'Fone Bluetooth';
const preco = 129.9;
const quantidade = 3;
const desconto = 0.15;
```

- 1. Título entre linhas de = (use **repeat**)
- 2. Rótulos à esquerda e valores à direita
- 3. Preço em reais e desconto em percentual

```
=====
FICHA DO PRODUTO
=====
Produto           Fone Bluetooth
Preço              R$ 129,90
Quantidade         3
Desconto           15%
=====
```

Desafio: mostre os mesmos dados com **console.table**.

COLETA DE INFORMAÇÕES

Lendo do teclado no Node

prompt()

Só existe no navegador. No Node dá ReferenceError.

Biblioteca

O npm tem pacotes prontos. Usamos o **@inquirer/prompts**.

await

A leitura é assíncrona: esperamos a resposta com **await**.

Sem criar interface nenhuma: **importamos e chamamos**.

COLETA DE INFORMAÇÕES

O que é uma biblioteca (pacote)

npm

Repositório público de pacotes (bibliotecas) para JavaScript.

package.json

Lista o nome do projeto e as bibliotecas que ele usa.

node_modules

Pasta onde os pacotes instalados ficam guardados.

Biblioteca = código pronto, escrito por outras pessoas, que reaproveitamos com **import**.

COLETA DE INFORMAÇÕES

Instalando a biblioteca

```
$ npm install @inquirer/prompts
```

```
{  
  "name": "aula1-sintaxe",  
  "type": "module",  
  "dependencies": {  
    "@inquirer/prompts": "^7.10.1"  
  }  
}
```

COLETA DE INFORMAÇÕES

Importando

```
import { input } from '@inquirer/prompts';  
import { input, number } from '@inquirer/prompts';  
import { input, number, confirm, select, checkbox, password } from '@inquirer/prompts';
```

import traz nomes de outro módulo.

Os nomes vão entre **chaves**.

Só funciona com **"type": "module"**.

COLETA DE INFORMAÇÕES

Como chamamos um prompt

```
const nome = await input({ message: 'Qual é o seu nome?' });
```

const nome

Guarda a resposta numa variável.

await

Espera a pessoa responder antes de seguir.

input({ ... })

Tipo da pergunta + objeto com as opções.

COLETA DE INFORMAÇÕES

Opções: um objeto { chave: valor }

```
const idade = await number({  
  message: 'Idade?',    // texto da pergunta  
  min: 0,               // menor valor aceito  
  max: 120,             // maior valor aceito  
  required: true,       // não aceita resposta vazia  
});
```

Objeto: pares **chave: valor**.

Separados por vírgula; a ordem não importa.

message é obrigatória.

COLETA DE INFORMAÇÕES

Tipos de prompt e o que devolvem

Função	Pergunta	Devolve
input	texto livre	string
number	um número	number
confirm	sim ou não	boolean
select	uma opção da lista	tipo do value
checkbox	várias opções	array de values
password	texto oculto	string

COLETA DE INFORMAÇÕES

input: texto livre

```
const nome = await input({ message: 'Nome?' });  
  
const cidade = await input({ message: 'Cidade?', default: 'São Paulo' });  
  
const login = await input({ message: 'Login?', required: true });
```

default: valor se apertar Enter.

required: recusa resposta vazia.

Sempre devolve **string**.

COLETA DE INFORMAÇÕES

number: valores numéricos

```
const idade = await number({  
  message: 'Idade?',  
  min: 0,  
  max: 120,  
  required: true,  
});
```

```
const preco = await number({ message: 'Preço?', step: 'any', required: true });
```

step: 'any' aceita decimais.

required: true evita **undefined**.

Vírgula decimal (**1,75**) é recusada.

Saudação interativa

1. Importe **input** e **number** de **@inquirer/prompts**
2. Pergunte o nome (**input**) e a idade (**number**, de 0 a 120)
3. Mostre: Olá, Ana! Ano que vem você terá 21 anos.
4. Mostre o **typeof** de cada resposta

Desafio: troque **number** por **input** na idade e veja o que acontece.

COLETA DE INFORMAÇÕES

confirm e password

```
const aceita = await confirm({ message: 'Aceita os termos?' });
const novidades = await confirm({ message: 'Receber novidades?', default: false });
console.log(aceita, typeof aceita); // true "boolean"

const senha = await password({ message: 'Senha?', mask: '*' });
console.log(typeof senha); // "string"
```

| **confirm** devolve **boolean**.

| Sem **default**, Enter significa **sim**.

| **password** esconde o que se digita.

COLETA DE INFORMAÇÕES

select: escolher uma opção

```
const cor = await select({  
  message: 'Cor favorita?',  
  choices: ['azul', 'verde', 'vermelho'],  
});  
  
console.log(cor, typeof cor); // verde "string"
```

Use ↑ ↓ e **Enter**.

Com textos simples, devolve o **próprio texto**.

Para outros tipos: próximo slide.

COLETA DE INFORMAÇÕES

select com valores de qualquer tipo

```
const parcelas = await select({
  message: 'Parcelas?',
  choices: [
    { name: 'À vista', value: 1 },
    { name: '3x sem juros', value: 3 },
    { name: '12x com juros', value: 12 },
  ],
});
console.log(parcelas, typeof parcelas); // 12 "number"
```

name: o que aparece na tela.

value: o que a função devolve.

value pode ser **number**, **boolean**,
texto...

COLETA DE INFORMAÇÕES

checkbox: várias opções

```
const extras = await checkbox({
  message: 'Extras?',
  choices: [
    { name: 'Embalagem', value: 'embalagem', checked: true },
    { name: 'Cartão', value: 'cartao' },
    { name: 'Entrega expressa', value: 'expressa' },
  ],
});
console.log(extras); // [ 'embalagem', 'expressa' ]
```

Devolve um **array** com os **value** marcados.

Espaço marca, Enter confirma.

checked: true já vem marcada.

Pedido de lanche

1. Importe **select**, **checkbox** e **confirm**
2. **select** do tamanho, com **value** numérico: P = 15, M = 20, G = 25
3. **checkbox** dos adicionais (bacon, queijo extra, ovo) e **confirm**: é para viagem?
4. Mostre o resumo, cada item com o seu **typeof**

Desafio: o que o **typeof** mostra para o array? Use **Array.isArray**.

COLETA DE INFORMAÇÕES

Validando a resposta

```
const email = await input({
  message: 'E-mail?',
  validate: (valor) => valor.includes('@') || 'Digite um e-mail válido',
});

const par = await number({
  message: 'Um número par?',
  required: true,
  validate: (n) => n % 2 !== 0 || 'Só números pares',
});
```

validate: true aceita, texto recusa.

O texto vira a **mensagem de erro**.

Por ora, use como **modelo**.

Convertendo respostas

```
const texto = await input({ message: 'Altura (m)? Ex.: 1,75' });  
const altura = Number(texto.replace(',', ' '));  
  
console.log(altura, typeof altura); // 1.75 "number"
```

Prefira **number()** quando quiser número.

Com **input**, converta com **Number()**.

Vírgula decimal: troque por ponto antes.

Altura com vírgula

1. Pergunte a altura com **input** (aceite **1,75**)
2. Converta para número e mostre o **typeof** antes e depois
3. Tente a mesma pergunta com **number**: o que acontece com **1,75** e com **1.75**?

Desafio: use **validate** para recusar resposta vazia.

Outros tipos de prompt (consulta)

Função	Como se responde	Devolve
rawlist	digita o número da opção	tipo do value
expand	tecla de atalho (y, n, a...)	tipo do value
search	digita para filtrar a lista	tipo do value
editor	abre um editor de texto	string

Mesmo modelo: **await função({ message, ... })**. Detalhes na documentação da biblioteca.

Outras formas de ler dados

Opção	Quando usar
@inquirer/prompts	Biblioteca do npm. Recomendada: é a que usamos.
readline/promises	Nativo do Node, sem instalar, mas exige criar a interface.
readline-sync	Síncrona (sem await), mas falhou com acentos no Windows.
process.argv	Dados na chamada: node app.js Ana 20.
prompt()	Só existe no navegador. No Node dá ReferenceError.

EXERCÍCIO 3 • REPRODUZA NO COMPUTADOR E TESTE

Cadastro com vários tipos

1. Importe as funções de que precisar de **@inquirer/prompts**
2. **input**: nome (obrigatório) e **number**: idade (0 a 120)
3. **number**: altura em metros (decimais!)
4. **confirm**: deseja receber novidades?
5. **select**: plano, com **value** numérico
6. **checkbox**: interesses; **password**: senha
7. Exiba cada resposta com o seu **typeof**

Desafio: mostre a senha como asteriscos (**repeat**) e use **console.table**.

INTERVALO

Pausa

Na volta: operações com as informações

OPERAÇÕES

Operadores aritméticos

```
7 + 2      // 9
7 - 2      // 5
7 * 2      // 14
7 / 2      // 3.5   (a divisão nunca é "inteira")
7 % 2      // 1     (resto da divisão)
2 ** 10    // 1024  (potência)
```

| / devolve decimal.

| % é o **resto** da divisão.

| ** é a potência.

OPERAÇÕES

Precedência dos operadores

```
2 + 3 * 4      // 14   (* antes de +)
(2 + 3) * 4     // 20   (parênteses primeiro)
10 - 2 - 3      // 5    (da esquerda para a direita)
2 ** 3 ** 2     // 512  (** vai da direita para a esquerda)
```

Parênteses primeiro.

******, depois ***** / **%**, depois **+** **-**.

Na dúvida, use **parênteses**.

OPERAÇÕES

Atribuição composta

```
let pontos = 10;  
pontos += 5;      // 15    (pontos = pontos + 5)  
pontos -= 3;      // 12  
pontos *= 2;      // 24  
pontos /= 4;      // 6  
pontos++;         // 7    (soma 1)  
pontos--;         // 6    (subtrai 1)
```

| **x += 5** é **x = x + 5**.

| **x++** soma 1, **x--** subtrai 1.

| Só vale para variáveis com **let**.

ATIVIDADE 8 • SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
7 / 2
```

```
7 % 3
```

```
2 + 3 * 4
```

```
(2 + 3) * 4
```

```
10 - 4 - 3
```

```
2 ** 5
```

```
15 % 4
```

```
9 / 0
```

```
let x = 10;
```

```
x += 5;
```

```
x *= 2;
```

```
x--;
```

```
// x vale?
```

1. Anote no caderno o resultado de cada linha
2. Confira com a turma (ou teste no REPL)
3. Nas que errou, explique o porquê

OPERAÇÕES

Convertendo tipos

```
Number('42')           // 42
Number('12px')          // NaN
parseInt('12px')        // 12
parseFloat('3.14abc')   // 3.14
String(42)              // "42"
Boolean('')             // false
```

Number é rigoroso (**NaN** se falhar).

parseInt/parseFloat leem o começo válido.

String e **Boolean** completam o quadro.

OPERAÇÕES

Conversões com valores especiais

```
Number('')           // 0      (texto vazio vira 0!)
Number(' 12 ')       // 12     (espaços nas pontas somem)
Number(true)         // 1
Number(null)         // 0
Number(undefined)    // NaN
+'5'                 // 5      (o + na frente também converte)
```

| **Number('')** dá **0**.

| **null** vira 0; **undefined** vira **NaN**.

| **+** na frente também converte.

OPERAÇÕES

Texto que parece número

```
'5' + 3           // "53"   concatena
'5' - 3           // 2      converte
'10' > '9'        // false  compara texto
Number('10') > Number('9') // true   compara número
```

O **+** concatena se houver texto.

Comparar textos $\neq$ comparar números.

Converta **na entrada**.

O bug da soma

```
import { input } from '@inquirer/prompts';

const a = await input({ message: 'Primeiro valor?' });
const b = await input({ message: 'Segundo valor?' });

console.log('Soma:', a + b);
```

1. Reproduza o código acima e rode com 20 e 5
2. Explique por que apareceu 205 em vez de 25
3. Conserte de duas formas: **Number()** e **number()**

Desafio: teste com 1.5 e 2.5 nas duas versões.

OPERAÇÕES

Operadores de comparação

```
5 > 3           // true
5 ≤ 3           // false
5 ≡ '5'         // false (valor E tipo)
5 = '5'         // true  (converte os tipos: evite)
0 = false       // true  (evite ==)
'a' < 'b'       // true  (ordem alfabética)
```

Devolvem sempre **true** ou **false**.

Use **sempre** **===** e **!==**.

== converte tipos: evite.

OPERAÇÕES

Operadores lógicos

```
const idade = 20;  
const temDocumento = true;  
  
idade ≥ 18 && temDocumento      // true   (E: os dois)  
idade < 18 || !temDocumento     // false  (OU: basta um)  
!(idade ≥ 18)                   // false  (NÃO: inverte)
```

&&: os dois precisam ser true.

||: basta um ser true.

! inverte o valor.

OPERAÇÕES

||, ?? e o operador ternário

```
const idade = 20;

const apelido = '' || 'sem apelido';    // || troca qualquer falsy
const pontos = 0 ?? 10;                 // ?? só troca null/undefined
const tipo = idade ≥ 18 ? 'adulto' : 'menor'; // ternário
```

|| troca qualquer valor **falsy**.

?? troca só **null** e **undefined**.

?: escolhe entre dois valores.

OPERAÇÕES

Truthy e falsy

Falsy (viram false)

false

0

''

null

undefined

NaN

Todo o resto é truthy

Inclusive '0', [] e {}.

Em &&, || e ?:, o JavaScript converte o valor para booleano.

Pode entrar?

1. Importe **number** e **confirm**; pergunte a idade, se tem ingresso e se está acompanhado
2. Regra: entra quem tem ingresso **E** (é maior de idade **OU** está acompanhado)
3. Mostre os booleanos e a frase "Entrada liberada" ou "Entrada barrada" com o operador ternário

Desafio: teste 4 combinações e confira se a regra funciona.

OPERAÇÕES

Operações com texto

```
const s = 'JavaScript';

s.length           // 10
s.toUpperCase()     // "JAVASCRIPT"
s.slice(0, 4)       // "Java"
s.indexOf('S')      // 4
s.includes('Script') // true
s.replace('Java', 'Type') // "TypeScript"
```

Métodos devolvem **novo** texto.

length não leva parênteses.

Posições começam em **0**.

OPERAÇÕES

Mais operações com texto

```
'Olá' + ' ' + 'mundo'      // "Olá mundo"
'ab'.repeat(3)              // "ababab"
'  oi  '.trim()             // "oi"
'a-b-c'.replaceAll('-', '+') // "a+b+c"
'JavaScript'.at(-1)         // "t"
'JavaScript'.startsWith('Java') // true
```

trim tira espaços das pontas.

at(-1) pega o último caractere.

replaceAll troca todas as ocorrências.

OPERAÇÕES

O objeto Math

```
Math.round(2.5)    // 3
Math.floor(3.9)    // 3    (para baixo)
Math.ceil(3.1)     // 4    (para cima)
Math.trunc(-3.7)   // -3    (corta os decimais)
Math.max(3, 9, 4)  // 9
Math.sqrt(16)      // 4
```

| **floor** para baixo, **ceil** para cima.

| **trunc** corta os decimais.

| **max/min** aceitam vários valores.

OPERAÇÕES

Math: mais funções

```
Math.abs(-5)           // 5
Math.min(3, 9, 4)      // 3
Math.PI                // 3.141592653589793
Math.random()          // sorteia de 0 até quase 1
Math.floor(Math.random() * 6) + 1 // um dado de 6 faces
```

Math.random() vai de 0 até quase 1.

Para inteiros: **floor** + * **N**.

Math.PI é a constante π .

Texto e sorteio

1. Pergunte o nome completo (**input**)
2. Mostre o tamanho, as maiúsculas, a primeira letra e o nome sem espaços nas pontas
3. Sorteie um dado de 6 faces e mostre: A, você tirou 3!

Desafio: sorteie um número de 10 a 20.

OPERAÇÕES

Precisão e valores especiais

```
0.1 + 0.2           // 0.30000000000000004
(0.1 + 0.2).toFixed(2) // "0.30"    (texto!)
7 / 0               // Infinity
0 / 0               // NaN
NaN === NaN         // false
Number.isNaN(NaN)   // true
```

| **0.1 + 0.2 ≠ 0.3.**

| **toFixed** devolve **texto**.

| **NaN** nunca é igual a nada.

EXERCÍCIO 4 · REPRODUZA NO COMPUTADOR E TESTE

Operações com números

1. Importe **number**; use **step: 'any'** e **required: true**
2. Média de três notas, com 1 casa decimal
3. Celsius para Fahrenheit: $F = C \times 9 \div 5 + 32$
4. IMC = peso ÷ altura ao quadrado
5. Segundos convertidos em horas, minutos e segundos

Desafio: mostre **true** ou **false** para "IMC entre 18,5 e 24,9" usando **&&**.

MINI-PROJETO

Recibo de compra no terminal

```
=====
RECIBO DE COMPRA
=====
Cliente           Ana Souza
Produto           Fone Bluetooth
Qty               3 x R$ 129,90
Subtotal          R$ 389,70
Desconto          R$ 38,97
Total             R$ 350,73
3x de             R$ 116,91
=====
```

- **Coleta:** cliente, produto, preço, quantidade, cupom, parcelas
- **Operações:** subtotal, desconto, total, parcela
- **Exibição:** recibo alinhado, em reais

MINI-PROJETO

Roteiro do projeto

1. Copie o esqueleto (próximo slide) e complete a coleta: **input**, **number**, **confirm** e **select**
2. Calcular subtotal, desconto (ternário), total e parcela
3. Formatar com **toLocaleString**, **padEnd** e **padStart**
4. Imprimir o recibo entre linhas de **=**
5. Testar com e sem cupom, e com preços decimais

Teste a cada passo antes de seguir para o próximo.

MINI-PROJETO

Esqueleto do projeto

```
import { input, number, confirm, select } from '@inquirer/prompts';

const brl = { style: 'currency', currency: 'BRL' };

const cliente = await input({ message: 'Cliente?', required: true });
const preco = await number({ message: 'Preço?', min: 0, step: 'any', required: true });
// TODO: produto, quantidade, cupom (confirm) e parcelas (select)

// TODO: subtotal, desconto (ternário), total e valor da parcela

// TODO: imprimir o recibo com padEnd, padStart e toLocaleString('pt-BR', brl)
```

FECHAMENTO

Resumo da aula

- **Regras:** ; comentários, nomes, const/let, tipagem dinâmica
- **Saída:** console.log, template literals, formatação
- **Entrada:** @inquirer/prompts e o tipo que cada função devolve
- **Operações:** aritméticos, conversões, comparação, lógica
- **Texto e números:** métodos de string e Math

FECHAMENTO

Próximos passos

Condicionais

if, **else** e **switch**.

Laços

for, **while** e **for...of**.

Funções

Declarar, chamar e reutilizar.

Coleções

Arrays e objetos.

Para casa: conversor de moedas: pergunte o valor (**number**) e a moeda (**select** com **value** = cotação) e exiba o resultado formatado.

FECHAMENTO

Para praticar em casa

Lista de questões

42 questões em 5 partes, com respostas comentadas no final.

Gabarito comentado

Atividades e exercícios resolvidos, com explicações nos comentários do código.

Como estudar

Tente antes, rode no Node e só então compare com o gabarito.

No fim da aula você recebe a **lista-de-questoes.md** e a pasta **gabarito/**. **Meta:** 2 questões de cada parte.

OBRIGADO

Dúvidas e onde estudar mais

Recomendado: W3Schools

Tutorial e referência de JavaScript com exemplos que você pode editar e executar. Conteúdo em inglês.

[Tutorial de JavaScript](#) · [Referência](#) · [Node.js](#)

MDN Web Docs (pt-BR): developer.mozilla.org/pt-BR

javascript.info: javascript.info

Node.js: nodejs.org/docs/latest/api

@inquirer/prompts: github.com/SBoudrias/Inquirer.js